

Project Report: Analysis of a Data Center Network

Author: Muhammad Omais Rana

1. Project Summary

This project involved modeling a 6-node data center network as a graph. We were given three files representing the network: `mat3.txt` (which nodes are connected), `cost3.txt` (the cost to send data over existing links), and `instal_cost3.txt` (the cost to install new cables).

The goal was to use graph algorithms to solve four distinct problems:

1. Find the cheapest routing cost between any two data centers.
 2. Find the cheapest way to install new cables to connect all data centers.
 3. Identify clusters (groups) of data centers.
 4. Create a new, simplified graph showing how the clusters connect.
-

2. Choice of Data Structures

For this C++ project, the following data structures were chosen:

- **2D Vectors:** This was the primary data structure. Since all input data was provided as $n \times n$ matrices, using 2D vectors was the most direct way to load, store, and process the adjacency, cost, and installation data.
 - **Edge Struct:** To use Kruskal's algorithm, it was easier to first convert the installation matrix into a list of edges. I used a simple struct `Edge { int src, dest, weight; }` to store all possible cable connections and their costs.
 - **Disjoint Set Union (DSU):** This structure is essential for Kruskal's algorithm. It provides a very fast way to check if adding a new cable (edge) would create a cycle.
 - **Stack (`std::stack<int>`):** Tarjan's algorithm for finding clusters requires a stack to keep track of the nodes visited during its Depth First Search.
-

3. Choice of Algorithms and Justification

An appropriate algorithm was selected for each of the project's four tasks.

Task 1: Shortest Path Routing

- **Algorithm: Floyd-Warshall Algorithm**
- **Justification:** The project asks for the shortest path between *any pair* of data centers, not just from a single source. Floyd-Warshall is the standard algorithm for this "All-Pairs Shortest Path" problem. It works perfectly with the 2D cost matrix we were given and correctly finds the minimum cost.

Task 2: Minimum Cost Cabling

- **Algorithm: Kruskal's Algorithm**
- **Justification:** The goal was to "link all data centers at minimum cost", without considering direction. This is the textbook definition of a **Minimum Spanning Tree (MST)**. I chose Kruskal's algorithm because its logic is straightforward.

Task 3: Identifying Clusters

- **Algorithm: Tarjan's Algorithm**
- **Justification:** The project defines a cluster as a group where any data center can route to any other data center *within that same group*. This is the exact definition of a **Strongly Connected Component (SCC)** in a directed graph. Tarjan's algorithm is a highly efficient method that uses a single DFS pass to find all SCCs.

Task 4: Graph of Clusters

- **Algorithm: Condensation Graph Construction**
 - **Justification:** This task was a direct implementation of the problem's description. I first used the cluster results from Tarjan's to create a map of which node belongs to which cluster. Then, I iterated through every original link (i, j) in mat3.txt. If node i was in Cluster X and node j was in a different Cluster Y, I incremented the count at N(X, Y).
-

4. Final Answers and Results

The C++ program produced the following results for the 6-node graph (indexed 0-5).

Answer 1: All-Pairs Shortest Path Matrix (Floyd-Warshall)

(Note: INF means no path exists)

```
0  2  3  9 10 12
1  0  1  7  8 10
INF INF  0  6  7  9
INF INF  5  0  1  3
INF INF  4 10  0  2
INF INF  2  8  9  0
```

Answer 2: Minimum Spanning Tree (Kruskal's)

- **Total Minimum Installation Cost:** 34
- **Edges to Install:**
 - (1, 3) with cost 1
 - (0, 3) with cost 8
 - (1, 2) with cost 8
 - (4, 5) with cost 8
 - (3, 4) with cost 9

Answer 3: Identified Clusters (Tarjan's SCCs)

- **Found 2 clusters:**
 - **Cluster 0 (X0):** { 5, 4, 3, 2 }
 - **Cluster 1 (X1):** { 1, 0 }

Answer 4: Cluster Adjacency Matrix N

(Shows 1 link from Cluster 1 to Cluster 0, and 0 links back)

```
N =  0  0
      1  0
```

Completed tasks6 / 6

Excellent! Your C++ graph algorithms project is now running successfully. Here's what was accomplished:

Project Setup Complete ✓
I've successfully compiled and executed your C++ program. The application implements four graph algorithms:

Task 1: Floyd-Warshall Algorithm (All-Pairs Shortest Path)

- Computed shortest paths between all pairs of data centers
- Shows the minimum cost to reach any center from any other center

Task 2: Kruskal's Algorithm (Minimum Spanning Tree)

- Total Minimum Installation Cost: 34
- Edges to install:
 - (1, 3) with cost 1
 - (0, 3) with cost 8
 - (1, 2) with cost 8
 - (4, 5) with cost 8
 - (3, 4) with cost 9

Task 3: Tarjan's Algorithm (Strongly Connected Components)

- Found 2 clusters:
 - Cluster 0 (X0): { 5, 4, 3, 2 }
 - Cluster 1 (X1): { 1, 0 }

Task 4: Condensation Graph (Matrix N)

- Built the cluster adjacency matrix showing connections between clusters
- Shows 1 edge from Cluster 1 to Cluster 0

./graph_prog...

--- Task 1: All-Pairs Shortest Path (Floyd-Warshall) ---

0	2	3	9	10	12
1	0	1	7	8	10
INF	INF	0	6	7	9
INF	INF	5	0	1	3
INF	INF	4	10	0	2
INF	INF	2	8	9	0

--- Task 2: Minimum Spanning Tree (Kruskal's) ---

Total Minimum Installation Cost: 34

Edges to install:

(1, 3) with cost 1

(0, 3) with cost 8

(1, 2) with cost 8

(4, 5) with cost 8

(3, 4) with cost 9

--- Task 3: Clusters (Tarjan's SCCs) ---

Found 2 clusters:

Cluster 0 (X0): { 5 4 3 2 }

Cluster 1 (X1): { 1 0 }

--- Task 4: Cluster Adjacency Matrix N ---

0	0
1	0

Figure 1. Screenshot of c++ program running in Replit compiler